Task Manager API

Complete Study Guide

Project	Task Manager REST API
Architecture	Clean Architecture (4 Layers)
Pattern	CQRS with MediatR
Framework	ASP.NET Core .NET 10
Database	SQL Server Express + EF Core
Author	Praveennath
Purpose	Portfolio Project — .NET Developer Role

Chapter 1 — What Is This Project?

This is a Task Manager REST API — a backend service that lets users create, read, update, and delete tasks. It is built using professional patterns that real companies use in production. The project is not just a tutorial — it is a portfolio piece designed to demonstrate enterprise-level .NET skills.

Why This Project Matters

Every company that hires .NET developers expects candidates to know Clean Architecture, CQRS, and EF Core. By building this project, you are showing employers that you write code the professional way — not just working code, but maintainable, testable, and scalable code.

What the API Does

Method	Endpoint	What It Does
GET	/api/Tasks	Returns all tasks from the database
GET	/api/Tasks/{id}	Returns one specific task by its ID
POST	/api/Tasks	Creates a new task and saves to database
PUT	/api/Tasks/{id}	Updates an existing task
DELETE	/api/Tasks/{id}	Deletes a task permanently

Chapter 2 — Clean Architecture

Clean Architecture is a way of organizing code so that each part of your application has one job and does not depend on things it does not need to know about. Think of it like a restaurant — the waiter does not cook, the chef does not take orders.

The 4 Layers

1. Domain	The heart of the app. Contains your business entities and interface contracts. No database. No HTTP. No framework. Just pure C# classes that represent your business rules. Everything else depends on this layer — but this layer depends on nothing.

Files: WorkTask.cs, BaseEntity.cs, IWorkTaskRepository.cs

2. Application	Where your features live. Every feature is either a Command (write something) or a Query (read something). Uses MediatR to route requests to the correct handler. This layer knows the Domain layer but does not know about databases or HTTP.

Files: CreateTaskCommand.cs, GetAllTasksQuery.cs, UpdateTaskCommand.cs, DeleteTaskCommand.cs

3. Infrastructure	Where the database lives. Contains the EF Core DbContext and the actual implementation of the repository interfaces defined in Domain. This is the only layer that writes SQL (through EF Core). Nothing else in the app knows SQL exists.

Files: AppDbContext.cs, WorkTaskRepository.cs, DependencyInjection.cs

4. API	The door of your application. Controllers receive HTTP requests and pass them to MediatR. That is all they do. They do not contain business logic. They do not touch the database directly.

Files: TasksController.cs, ExceptionHandlingMiddleware.cs, Program.cs

The Golden Rule

Each layer only knows about the layer directly below it. API knows Application. Application knows Domain. Domain knows nothing. This means you can change the database from SQL Server to PostgreSQL by only touching the Infrastructure layer — nothing else changes.

Chapter 3 — CQRS and MediatR

CQRS stands for Command Query Responsibility Segregation. It means: separate the code that reads data from the code that writes data. Every action in your app is either a Command or a Query — never both.

Commands vs Queries

	Command	Query
Purpose	Write / change data	Read data only
Examples	Create, Update, Delete	GetAll, GetById
Returns	The created/updated object	Data or list of data
Rule	Never returns a Query	Never changes data

How MediatR Works

MediatR is the messenger. Instead of the controller calling the handler directly, it sends a request to MediatR and MediatR finds the right handler automatically. This means the controller never knows the handler exists — they are completely decoupled.

Flow of a Create Task request:

Step	Who	What happens
1	Swagger / Client	Sends POST /api/Tasks with JSON body
2	TasksController	Receives the request, calls _mediator.Send(command)
3	MediatR	Finds CreateTaskCommandHandler automatically
4	CreateTaskCommandHandler	Builds a WorkTask object from the command data
5	WorkTaskRepository	Calls EF Core to INSERT into SQL Server
6	SQL Server	Saves the row, returns the new ID
7	Response	Travels back through all layers as clean JSON

Chapter 4 — Entity Framework Core

Entity Framework Core (EF Core) is an ORM — Object Relational Mapper. It translates your C# code into SQL automatically. You write C# classes; EF Core creates database tables. You write C# queries; EF Core writes the SQL.

Key Concepts

DbContext — The main class that manages the database connection. AppDbContext is your DbContext. It holds DbSet properties that map to database tables.

DbSet — Represents a table in the database. DbSet Tasks means there is a Tasks table in SQL Server that maps to the WorkTask class.

Migrations — Files that describe how to create or update your database schema. When you run 'dotnet ef migrations add' it creates a C# file describing the table changes. When you run 'dotnet ef database update' it applies those changes to SQL Server.

SaveChangesAsync() — The method that actually sends SQL to the database. EF Core holds your changes in memory until you call this. Nothing hits the database before this call.

Fluent API — The configuration in OnModelCreating that defines table rules — primary keys, required fields, max lengths, relationships.

What EF Core Does Behind the Scenes

Your C# Code	SQL EF Core Generates
`_context.Tasks.ToListAsync()`	`SELECT * FROM Tasks`
`_context.Tasks.FindAsync(id)`	`SELECT * FROM Tasks WHERE Id = @id`
`_context.Tasks.Add(task)`	`INSERT INTO Tasks (Title...) VALUES (...)`
`_context.Tasks.Update(task)`	`UPDATE Tasks SET Title=... WHERE Id=@id`
`_context.Tasks.Remove(task)`	`DELETE FROM Tasks WHERE Id=@id`

Chapter 5 — Every File Explained

Domain Layer

BaseEntity.cs

The base class every entity inherits from. Provides Id, CreatedAt, and UpdatedAt automatically. Marked abstract so you can never create a BaseEntity directly — only inherit from it.

WorkTask.cs

Your main entity. Represents one task in the system. Inherits Id/CreatedAt/UpdatedAt from BaseEntity. Adds Title, Description, IsCompleted, DueDate. Named WorkTask instead of Task because Task is a reserved C# keyword.

IWorkTaskRepository.cs

An interface — a contract. Defines what any task repository must be able to do: GetAll, GetById, Create, Update, Delete. The Domain layer defines the contract; Infrastructure implements it. This is Dependency Inversion in action.

Application Layer

Result.cs

A wrapper class for all responses. Instead of returning raw data or throwing exceptions, every handler returns Result which is either Success(data) or Failure(errorMessage). This makes every response consistent and predictable.

GetAllTasksQuery.cs

A Query that asks for all tasks. The record carries no data (no parameters needed). The Handler calls GetAllAsync on the repository and wraps the result in Result.Success.

GetTaskByIdQuery.cs

A Query that asks for one task by Id. If not found, returns Result.Failure('Task not found') instead of crashing. This is safe null handling.

CreateTaskCommand.cs

A Command that creates a new task. Carries Title, Description, DueDate. Handler builds a WorkTask object and calls CreateAsync on the repository.

UpdateTaskCommand.cs

A Command that updates an existing task. Fetches the task first, applies the new values, stamps UpdatedAt, then saves. Never updates blindly.

DeleteTaskCommand.cs

A Command that deletes a task. Fetches first to confirm it exists, then deletes. Returns Result because there is nothing to return after deletion.

DependencyInjection.cs

Registers MediatR with the DI container. Assembly scanning finds all handlers automatically — you never register them one by one.

Infrastructure Layer

AppDbContext.cs

The EF Core database context. Holds DbSet Tasks which maps to the Tasks table. OnModelCreating defines rules: Id is primary key, Title is required max 200 chars.

WorkTaskRepository.cs

The actual implementation of IWorkTaskRepository. This is where real database operations happen using EF Core. GetAllAsync, GetByIdAsync, CreateAsync, UpdateAsync, DeleteAsync — all implemented here.

DependencyInjection.cs

Registers AppDbContext with SQL Server connection string. Registers WorkTaskRepository as the implementation of IWorkTaskRepository. Used as an extension method so Program.cs can call services.AddInfrastructure() cleanly.

AppDbContextFactory.cs

Used only at design time during migrations. Tells EF Core how to build the DbContext when running dotnet ef commands without starting the full application.

API Layer

TasksController.cs

The entry point for HTTP requests. Has 5 endpoints: GET all, GET by id, POST, PUT, DELETE. Each method calls _mediator.Send() and returns the result as HTTP response. No business logic here — only routing requests to MediatR.

ExceptionHandlingMiddleware.cs

Wraps every request in a try/catch. If anything crashes anywhere in the app, this catches it, logs it, and returns a clean JSON error response instead of a raw crash.

Program.cs

The startup file. Registers all services (AddApplication, AddInfrastructure), sets up Swagger, configures the middleware pipeline, and starts the web server.

appsettings.json

Configuration file. Holds the SQL Server connection string. Never hardcode connection strings in code — always read from configuration.

Chapter 6 — Full File Structure

```
D:\TaskManager\
■
■■■ TaskManager.sln
■
■■■ TaskManager.Domain\
■   ■■■ Common\
■   ■   ■■■ BaseEntity.cs
■   ■■■ Entities\
■   ■   ■■■ WorkTask.cs
■   ■■■ Interfaces\
■       ■■■ IWorkTaskRepository.cs
■
■■■ TaskManager.Application\
■   ■■■ Common\
■   ■   ■■■ Result.cs
■   ■■■ Features\
■   ■   ■■■ Tasks\
■   ■       ■■■ Commands\
■   ■       ■   ■■■ CreateTaskCommand.cs
■   ■       ■   ■■■ UpdateTaskCommand.cs
■   ■       ■   ■■■ DeleteTaskCommand.cs
■   ■       ■■■ Queries\
■   ■           ■■■ GetAllTasksQuery.cs
■   ■           ■■■ GetTaskByIdQuery.cs
■   ■■■ DependencyInjection.cs
■
■■■ TaskManager.Infrastructure\
■   ■■■ Persistence\
■   ■   ■■■ AppDbContext.cs
■   ■■■ Repositories\
■   ■   ■■■ WorkTaskRepository.cs
■   ■■■ DependencyInjection.cs
■
■■■ TaskManager.API\
    ■■■ Controllers\
    ■   ■■■ TasksController.cs
    ■■■ Middleware\
    ■   ■■■ ExceptionHandlingMiddleware.cs
    ■■■ Properties\
    ■   ■■■ launchSettings.json
    ■■■ AppDbContextFactory.cs
    ■■■ Program.cs
    ■■■ appsettings.json
    ■■■ appsettings.Development.json
```

Chapter 7 — Tools and NuGet Packages

Tool / Package	Layer	Purpose
ASP.NET Core	API	Web framework — handles HTTP requests and routing
MediatR	Application	Routes commands and queries to the correct handler
FluentValidation	Application	Validates incoming data with clean rule-based syntax
Entity Framework Core	Infrastructure	ORM — translates C# to SQL automatically
EF Core SqlServer	Infrastructure	EF Core driver specifically for SQL Server
EF Core Tools	Infrastructure	Enables dotnet ef migration commands
EF Core Design	API	Required for migrations to work from the API project
Swashbuckle	API	Generates Swagger UI for testing the API in the browser
SQL Server Express	Database	Free local SQL Server instance — same as production
xUnit	Tests	Unit testing framework for testing individual handlers

Chapter 8 — Key Commands to Remember

Build the solution

```
dotnet build
```

Run the API

```
dotnet run --project TaskManager.API
```

Add a migration

```
dotnet ef migrations add MigrationName --project TaskManager.Infrastructure
--startup-project TaskManager.API
```

Apply migrations to DB

```
dotnet ef database update --project TaskManager.Infrastructure --startup-project
TaskManager.API
```

Remove last migration

```
dotnet ef migrations remove --project TaskManager.Infrastructure --startup-project
TaskManager.API
```

Add a NuGet package

```
dotnet add ProjectName package PackageName
```

Add project reference

```
dotnet add ProjectA reference ProjectB
```

Open in VS Code

```
code .
```

Create solution file

```
dotnet new sln -n SolutionName
```

Add project to solution

```
dotnet sln add ProjectName
```

Chapter 9 — What You Built and What It Proves

You built this project from scratch — every file, every line, manually. No scaffolding. No copy-paste from tutorials. This matters.

Skills You Demonstrated

Clean Architecture — You know how to separate concerns into layers and why it matters

CQRS Pattern — You understand the difference between Commands and Queries and why they are separated

MediatR — You know how to decouple request senders from handlers using a mediator

Entity Framework Core — You can map C# classes to database tables, write migrations, and perform CRUD operations

Dependency Injection — You understand how .NET's built-in DI container works and how to register services

Repository Pattern — You know how to abstract database access behind interfaces

REST API Design — You built proper HTTP endpoints following REST conventions

Global Error Handling — You know how to catch all exceptions in one place and return clean responses

SQL Server — You connected a real database, created tables from code, and saved real data

Git / GitHub — You version controlled your project and published it as a portfolio piece

What Comes Next

1. Add JWT authentication — secure the API so only logged-in users can access tasks
2. Add FluentValidation rules — validate that Title is not empty, DueDate is in the future
3. Add pagination — return tasks 20 at a time with page number support
4. Add unit tests with xUnit — test each handler independently
5. Add Blazor frontend — build a web UI in C# that calls this API
6. Deploy to Railway free tier — make the API publicly accessible online
7. Add GitHub Actions CI/CD — automatically build and test on every push

---

You started this project as a beginner with zero .NET experience. You now have a production-pattern API running on your machine, saved in GitHub, and a deep understanding of every line of code inside it. That is real progress.